CP 2026 Assignment 1 [7 Marks]

Date / Time	23 March 2026 – 5 April 2026 23:59 PM
Course	**[M1522.600] Computer Programming**
Instructor	Youngjae Yu

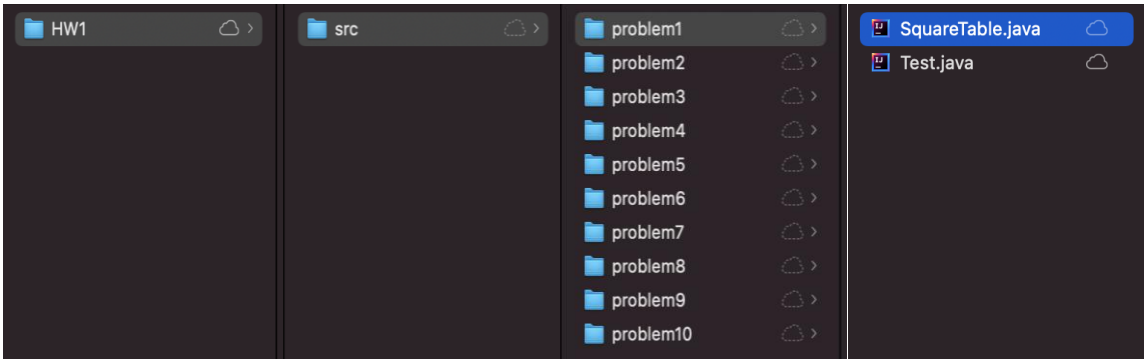
- You can refer to the Internet or other materials to solve the assignment, but you *__SHOULD NOT__* discuss the question with anyone else and need to code **ALONE**.
- We will **use the automated copy detector to check the possible plagiarism** of the code between the students. The copy checker is reliable so it is highly likely to mark a pair of codes as the copy even though two students quickly discuss the idea without looking at each other's code. Of course, we will evaluate the similarity of a pair compared to the overall similarity for the entire class.
- We will do the manual inspection of the code. In case we doubt that the code may be written by someone else, we reserve the right to request an explanation about the code. We will ask detailed questions that cannot be answered if the code is not written by yourself.
- For the first event of plagiarism, you will get 0 marks for the specific assignment. You will fail the course and be reported to the dean if you copy others' code more than once.
- Download and unzip "HW1.zip" file from the etl. The file contains the skeleton codes for 10 questions (in "/problemX" directory for question #X).
- **Do not modify the overall directory structure**. Simply fill in the codes in the appropriate files.

- Do not use external libraries.
- Do not use Java Collections Framework (i.e., ArrayList, Hashmap, ...). (https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html)
- If you have any questions, feel free to ask any in the Slack channel.

Submission Guidelines

1. For submission, compress the entire "HW1" directory into a single zip file with name "{your student id}.zip" (ex: 2026-12345.zip).
2. The directory structure of your submitted file (after unzipping) should look like the following. When you extract the zip file, **there must be the HW1/ directory.**

   Note that you will get **0 marks** for the wrong submission format.



3. Submit the zip file on etl.

Overview

1. The goal of this assignment is to implement simple Java programs.
2. Implement the **Target function** indicated in each question. **Do NOT change the directory structure, existing file names, existing class names, or existing function signatures.** You may add new files, classes, or functions if necessary.
3. Assume all inputs are correct. This means that you do not have to handle wrong inputs.
4. The trailing spaces after the last line of the output will be ignored. However, the intermediate lines should NOT have trailing spaces when output has multiple lines.

Answer	Student Output	Is it correct?
abc	abc	O
abc abc	abc abc	O
abc abc	abc abc	X

5. We will use a comprehensive set of test cases for evaluation. So we strongly recommend that you use test cases of your own. You can easily test your code by adding test cases in the Test.java file.
6. There is **NO partial point** for a question. That means the final score for each question will be either 0 or 1.

Question 1: Square Table [0.7 Marks]

Objective: Write a program that prints out all squares that are greater than or equal to the first input number n and less than or equal to the second input number m.

Target Function: `public static void printSquareTable(int n, int m)` (in SquareTable.java)

Input: int n, m (1 <= n <= m <= 900).

Output: Square table that corresponds to input numbers N and M (see the example below). The output format is **'k times k = k^2'**.

Note: As mentioned in the overview, the intermediate lines should **NOT** have trailing spaces!

Input	Output
n=25 m=49	5 times 5 = 25 6 times 6 = 36 7 times 7 = 49
n=1 m=23	1 times 1 = 1 2 times 2 = 4 3 times 3 = 9 4 times 4 = 16
n=3 m=3	

Question 2: Sum of Even Fibonacci Numbers [0.7 Marks]

Objective: Write a program that prints the first N Fibonacci numbers and the sum of the even Fibonacci numbers among them.

Description: Fibonacci Numbers, commonly denoted as F_n, form a sequence called the Fibonacci sequence. The sequence starts with the two numbers, 0 and 1, and each number in the sequence is the sum of its two preceding ones, that is, $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ for $n > 1$. Print $F_0 \sim F_{N-1}$ given N as the input.

Reference: https://en.wikipedia.org/wiki/Fibonacci_number

Target Function: `public static void printFibonacciNumbers(int n)` (in FibonacciNumbers.java)

Input: Integer n ($1 <= n <= 46$).

Output: The first N Fibonacci Numbers from F_0 to F_{n-1}. On the next line, print the sum of the even Fibonacci numbers. If the sum has more than five digits, print only its last five digits (including leading zeros).

Hint: Be careful with the types!

Input	Output
5	0 1 1 2 3 even sum = 2
8	0 1 1 2 3 5 8 13 even sum = 10
30	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75025 121393 196418 317811 514229 even sum = 57114

Question 3: Drawing Figure [0.7 Marks]

Objective: Write a program that prints the appropriate star(*) pattern for a given number.

Target Function: `public static void drawFigure(int n)` (in DrawingFigure.java)

Input: Integer n (1 <= n <= 50).

Output: The corresponding star pattern as below.

Note: As mentioned in the overview, the intermediate lines should **NOT** have trailing spaces!

Input	Output
1	*
2	*** *

3	*****
	  * *
	  *

4	*******
	  * *
	  * *
	  *

Question 4: Character Finder [0.7 Marks]

Objective: Write a program that

1. Takes an arbitrary string composed of alphabets.
2. Prints all alphabets appearing in the string and the index of their first & last occurrence. Print them in lexicographic order (A, B, C, …, Y, Z, a, b, c, …, y, z). Do NOT print out alphabets that have never appeared in the input string. See the examples below.
3. The index starts with 0. (The first character of the string has index 0.)

Target Function: `public static void findCharacter(String str)` (in CharacterFinder.java)

Input: String str (1 <= str.length() <= 200) composed of alphabets.

Output: For every appearing alphabet **x**, print **"x: i j"** in a line where x is the alphabet and i, j is the index of the first and the last occurrences, respectively.

Note: As mentioned in the overview, the intermediate lines should **NOT** have trailing spaces!

Input	Output
abb	a: 0 0 b: 1 2
bbbaAabaccHadAHbcbHHAdcbbeHaH	A: 4 20 H: 10 28 a: 3 27 b: 0 24 c: 8 22 d: 12 21 e: 25 25

Question 5: Descending Numbers[0.7 Marks]

Objective: Write a program that counts the number of valid ways to separate the given string.

Description:

You wrote down many positive integers in a string called num. However, you forgot to add commas to separate the numbers.

You remember the following:

- The list of integers was **strictly decreasing**.
- Every separated number was a **positive integer**.
- No separated number had **leading zeros**.

Return the number of possible lists of integers that could have formed the string num. If the answer has more than five digits, print only its last five digits (including leading zeros).

Target Function: public static void countValidAddress(String str) (in IPAddress.java)

Input: String str (1 <= str.length() <= 99) which is composed only of digits.

Output: The number of valid ways to separate num into a sequence of strictly decreasing positive integer. If the answer has more than five digits, print only its last five digits (including leading zeros).

Input	Output	Explanation
321	3	The valid ways are “321” “32,1” “3,2,1”
6543	5	The valid ways are 6543 “654,3” “65,43” “65,4,3” “6,5,4,3”
050	0	No valid split exists

Question 6: Prime Factorization [0.7 Marks]

Objective: Write a program that factorizes the given integer into prime numbers in exponent form.

Description: A prime number (or a prime) is a natural number greater than 1 that is not a product of two smaller natural numbers. The smallest prime number is 2, and the second smallest prime is 3. Prime factorization is finding which prime numbers multiply together to make the given number. For example, 12 can be factorized into 2 x 2 x 3, which can be written as 2^2 x 3.

Reference:

https://en.wikipedia.org/wiki/Prime_number

https://en.wikipedia.org/wiki/Integer_factorization

Target Function: `public static void factorize(int n)` (in PrimeFactorization.java)

Input: Integer n (1 < n <= 1000).

Output: Print the prime factorization of n in increasing order of prime factors.

- If a prime factor appears more than once, print it in the form p^k.
- If a prime factor appears only once, print only p
- Separate factors using " x "

Input	Output
12	2^2 x 3
13	13
100	2^2 x 5^2

Question 7: Palindrome String [0.7 Marks]

Objective: Write a program that counts the number of palindrome substrings in a given string.

Description: A palindrome is a string that is the same when you read it forward from the beginning or backward from the end. For example, "abcba" is a palindrome and "abc" is not a palindrome. Given a string, count how many of unique substrings are palindromes.

Target Function: `public static void printPalindromeCount(String str)` (in PalindromeString.java)

Input: String str (1 <= str.length() <= 100) which consists of only alphabets and digits.

Output: T Print the number of distinct palindrome substrings in the given string. If the same palindrome appears multiple times at different positions, count it only once.

Input	Output	Explanation
abc	3	the distinct palindrome substrings are "a", "b", "c"
aaa	3	the distinct palindrome substrings are "a", "aa", "aaa"
abba	4	the distinct palindrome substrings are "a", "b", "bb", "abba"

Question 8: Matrix Multiplication [0.7 Marks]

Objective: Write a program that runs matrix multiplication with the two input matrices(if multiplication is possible).

Description: Implement a method that prints out the result of the matrix multiplication. The output format should be as follows.

- Each row of the result matrix should be printed in one line.
- The numbers of each row should be separated by whitespace.

Reference: https://www.mathsisfun.com/algebra/matrix-multiplying.html

Target Function: `public static void multiply(int[][] A, int[][] B)` (in MatrixMultiplication.java)

Input: Two 2D int arrays of size N x M and P x K (1 <= N, M, K, P <= 8).

Output: Result matrix of the matrix multiplication. If multiplication is not possible, print "invalid"

Note: As mentioned in the overview, the intermediate lines should **NOT** have trailing spaces!

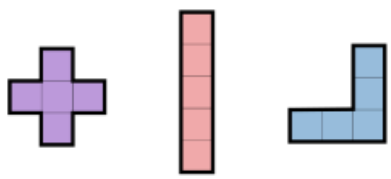
Input	Output
A = [[1 2] [3 4] [5 6]] B = [[1 1 2] [1 0 0]]	3 1 2 7 3 6 11 5 10
A = [[1 2 3] [4 5 6]] B = [[1 2] [3 4]]	invalid

Question 9: PentominoSum [0.7 Marks]

Objective: Write a program that calculates the maximum sum of the numbers that one pentomino covers.

Description: A pentomino is a shape where five 1x1 squares are connected together. You are trying to place a pentomino on a board of size N x M. Each cell of the board has an integer written on it. When you place a pentomino on the board, five numbers should overlap with the pentomino. Write a program that finds out the maximum sum of the numbers that can overlap with a single pentomino. A pentomino may be rotated or flipped.

Use the following three pentominoes.



Target Function: `public static void printMaxSum(int[][] board)` (in PentominoSum.java)

Input: 2D array of integers of size N x M (5 <= N, M <= 10).

Output: The maximum sum.

Input	Output
[[1 1 1 1 1] [1 4 4 5 1] [1 2 1 4 1] [1 2 1 4 1] [1 1 1 1 1]]	21

Explanation: The maximum sum can be obtained by placing ㄱ shaped pentomino as follows.

1	1	1	1	1
1	4	4	5	1
1	2	1	4	1
1	2	1	4	1
1	1	1	1	1

Question 10: Secret Telling [0.7 Marks]

Objective: Write a program that prints the number of people who know the secret at day n.

Description: On day 1, one person knows the secret. Two parameters are given: wait and forget. Each person tells the secret to a new person every day, starting from wait days after hearing the secret. Each person forgets the secret forget days after hearing the secret. On the day of forgetting the secret, the person cannot tell the secret to a new person.

Target Function: `public static void numSecretKnowing(int n, int wait, int forget)` (in SecretTelling.java)

Input: Integer n, wait, forget (1 <= n <= 20, 1 <= wait, forget <= n)

Output: The number of people knowing the secret.

Input	Output
n=6, wait=2, forget=4	5

Explanation: Let the name of people P1, P2, P3, ...

Day	What happens	Who knows secret
1	P1 knows the secret.	P1
2	P1 knows the secret.	P1
3	P1 tells P2.	P1, P2
4	P1 tells P3.	P1, P2, P3
5	P1 forgets. P2 tells P4.	P2, P3, P4
6	P2 tells P5. P3 tells P6.	P2, P3, P4, P5, P6